

CG2271 Real-Time Operating Systems

Tutorial 4

Introduction to POSIX Threads

This tutorial is structured as a set of experiments to get you familiar with using POSIX threads.

Question 1 Introduction and Parameter Passing

We begin with how to create POSIX threads, and also how to pass parameters to thread functions.

In POSIX, threads are written as functions. These functions can be nested, i.e. they can call other functions, and these will all be counted as part of the same thread.

Create a file called q1.c and key in the following:

We begin first by importing pthread.h, which contains the signatures for the pthread functions:

```
#include <stdio.h>
#include <pthread.h>
```

We next create the thread function:

```
void *t1(void *p) {
    int ndx = (int) p;

    printf("I am thread %d\n", ndx);
    pthread_exit(NULL);
}
```

Some explanations:

- Thread functions are always defined as returning void * (the value returned to pthread_join), taking one argument also of type void *:

```
void *t1(void *p) {
```

In C, void * is used to define the function return type, parameter type, or even variable type when the type is not known ahead of time.

- We can extract out the arguments passed to the thread function via parameter p, by casting it to the type that we expect. In this case we cast it into an int:

```
int ndx = (int) p;
```

The rest of the thread function is self-explanatory.

We go on to define main:

```
#define NUM_THREADS 10
int main() {

    int i;
    pthread_t tid[NUM_THREADS];

    for(i = 0; i<NUM_THREADS; i++) {
        pthread_create(&tid[i], NULL, t1, (void *) i);
    }

    for(i=0; i<NUM_THREADS; i++)
        pthread_join(tid[i], NULL);

    printf("All threads done!\n");

    return 0;
}
```

Explanations:

- We declare a macro called #NUM_THREADS to set the number of threads. This isn't strictly necessary but it is good programming practice to set all constants as macros at the start of your program.
- We create an array to store the thread IDs, which will be needed later:

```
int i;
pthread_t tid[NUM_THREADS];
```

- We now create the threads. Note how we pass our argument i – We cast it to (void *), which is the type for the thread function parameter p (see above). Recall that in the thread function we cast it back to int.

```
for(i = 0; i<NUM_THREADS; i++) {
    pthread_create(&tid[i], NULL, t1, (void *) i);
}
```

- Finally we call pthread_join so that main will wait for all threads to complete. If we don't do this, main may exit before the threads finish.

```
for(i=0; i<NUM_THREADS; i++)
    pthread_join(tid[i], NULL);

printf("All threads done!\n");
```

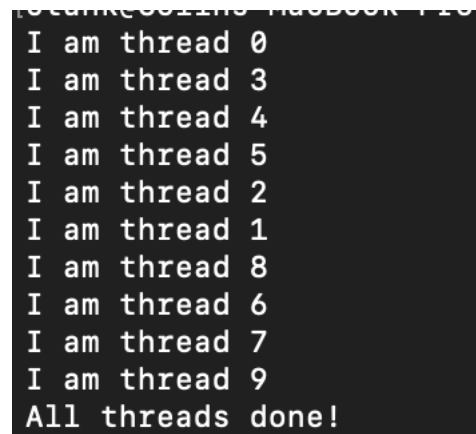
We compile our program using:

```
gcc -pthread q1.c -o q1
```

And run it using:

```
./q1
```

Our output will look like this:

A terminal window with a dark background and light-colored text. The text shows the output of a program with 10 threads. The threads print their IDs in a non-sequential order: 0, 3, 4, 5, 2, 1, 8, 6, 7, 9. Finally, the message 'All threads done!' is printed.

```
I am thread 0  
I am thread 3  
I am thread 4  
I am thread 5  
I am thread 2  
I am thread 1  
I am thread 8  
I am thread 6  
I am thread 7  
I am thread 9  
All threads done!
```

If you run it several times you will notice that the threads do not always run in the same order. I.e. the sequence of thread execution is not guaranteed.

Question 2 Finding the Maximum Value of a Large Array

In this question we will look at how we can apply pthreads to solving a simple toy problem. We have a large array of 1,000,000 random integers, and we will divide this amongst 10 threads. Each thread will look for the largest element of its sub-section, then return it to main which will find the largest of all the largest elements.

Create a file called q2.c and key in what is shown.

We begin by including the headers we need. We include `stdlib.h` which lets us access the random number generator `rand()` and to seed it using `srand()`, and we also include `time.h` to access the `time()` function. We also declare our array, length of our array, and the number of threads we plan to spawn:

```

#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>
#include <time.h>

#define ARRAY_LEN 1000000

int array[ARRAY_LEN], actual_max = -1;

#define NUM_THREADS 10

```

We now create the function to populate the array with numbers between 0 and 1,000,000:

```

void populate_array() {
    int i;

    srand(time(NULL));
    for(i=0; i<ARRAY_LEN; i++){
        array[i] = rand() % 10000000;

        if(array[i] > actual_max)
            actual_max = array[i];
    }
}

```

Notice how we seed the random number generator using srand: The time function returns the current time in terms of the number of seconds elapsed since 1 January 1970, 00:00 UTC, which is constantly changing. Thus this effectively gives the random number generator a “random” starting value, letting us generate a different sequence of numbers each time we run the program.

To check the correctness of our program, our populate_array function finds the actual maximum and stores it in actual_max.

We now create the actual thread function:

```

void *findmax(void *myindex) {
    int ndx = (int) myindex;
    int elts_per_thread = ARRAY_LEN / NUM_THREADS;

    int start = ndx * elts_per_thread;
    int end = start + elts_per_thread;
    int max = -1, i;

    for(i=start; i<end; i++)
        if(array[i] > max)
            max = array[i];

    printf("Largest value found by thread %d is %d\n", ndx, max);
    pthread_exit((void *) max);
}

```

- Our plan is to divide the array into 10 parts. Each part would have $1,000,000 / 10 = 100,000$ elements

Thread 0 will take the first part (0 – 99,999), thread 1 will take the second part (100,000 – 199,999), thread 2 will take the third part (200,000 – 299,999) etc. Thus a thread with index number *ndx* would take $ndx \times 100000$ to $ndx \times 100000 + 100000$. So in the first part we figure out the thread's index, the number of elements each thread needs to work on, and the start and end indices:

```
int ndx = (int) myindex;
int elts_per_thread = ARRAY_LEN / NUM_THREADS;

int start = ndx * elts_per_thread;
int end = start + elts_per_thread;
```

- We then go on to find the actual maximum:

```
for(i=start; i<end; i++)
    if(array[i] > max)
        max = array[i];
```

- We use `pthread_exit` to pass the result back. Note that the result must be cast to `void *` when passing it.

```
printf("Largest value found by thread %d is %d\n", ndx, max);
pthread_exit((void *) max);
```

We now create our main:

```
int main() {
    int i;
    pthread_t tid[NUM_THREADS];

    populate_array();
    for(i=0; i<NUM_THREADS; i++) {
        pthread_create(&tid[i], NULL, findmax, (void *) i);
    }

    int curr_max = -1;
    void *ret = 0;

    for(i=0; i<NUM_THREADS; i++) {
        pthread_join(tid[i], &ret);

        if((int) ret > curr_max)
            curr_max = (int) ret;
    }

    printf("All threads done. Maximum is %d. Actual maximum is %d\n", curr_max, actual_max);
    if(curr_max == actual_max)
        printf("We found the correct maximum!\n");

    return 0;
}
```

As before we create an array `tid` to store the thread IDs, then call `pthread_create`, and pass in “`i`” as the thread index. As seen earlier the threads will then use this to figure out which parts of the array to work on.

```
for(i=0; i<NUM_THREADS; i++) {
    pthread_create(&tid[i], NULL, findmax, (void *) i);
}
```

We now need to read the values returned by each thread. To do this, we create a variable called “`ret`” of type `void *`, and pass in a pointer to this variable to `pthread_join`:

```
int curr_max = -1;
void *ret = 0;

for(i=0; i<NUM_THREADS; i++) {
    pthread_join(tid[i], &ret);

    if((int) ret > curr_max)
        curr_max = (int) ret;
}
```

This is used to compare against the current largest value that we currently have from the threads. If the new value is larger, it will become the new largest value.

Finally we print out the largest value found in the entire array:

```
printf("All threads done. Maximum is %d. Actual maximum is %d\n", curr_max, actual_max);
if(curr_max == actual_max)
    printf("We found the correct maximum!\n");
```

Compile the program using:

```
gcc -pthread q2.c -o q2
```

And run using:

```
./q2
```

You will see an output similar to this:

```
Largest value found by thread 0 is 9999775
Largest value found by thread 1 is 9999983
Largest value found by thread 6 is 9999796
Largest value found by thread 2 is 9999984
Largest value found by thread 4 is 9999820
Largest value found by thread 5 is 9999898
Largest value found by thread 9 is 9999746
Largest value found by thread 8 is 9999500
Largest value found by thread 7 is 9999961
Largest value found by thread 3 is 9999898
All threads done. Maximum is 9999984. Actual maximum is 9999984
We found the correct maximum!
```

Question 3. Race Conditions

We will now explore the issue of race conditions and how to fix them. Create a program called q3.c, and key in the following code:

```
#include <stdio.h>
#include <pthread.h>

int sum = 0;
void *addone(void *p) {
    for(int i=0; i<10000; i++)
        sum++;

    pthread_exit(NULL);
}

int main() {

    int i;

    pthread_t tid[10];
    for(i=0; i<10; i++)
        pthread_create(&tid[i], NULL, addone, NULL);

    for(i=0; i<10; i++)
        pthread_join(tid[i], NULL);

    printf("Final value of sum = %d\n", sum);

    return 0;
}
```

This code is fairly self-explanatory. Only two things stand out from the previous two questions:

- We update a global variable called sum.

```
int sum = 0;
void *addone(void *p) {
    for(int i=0; i<10000; i++)
        sum++;

    pthread_exit(NULL);
}
```

- We don't use the void *p parameter in the thread function, hence when we call pthread_create, we pass in NULL:

```
pthread_t tid[10];
for(i=0; i<10; i++)
    pthread_create(&tid[i], NULL, addone, NULL);
```

Compile using:

```
gcc -pthread q3.c -o q3
```

Run using:

```
./q3
```

Since we have 10 threads each increasing sum by 10,000 times, our final answer should be 100,000. However we can run it many times and (almost) never get there:

```
ctank@Colins-MacBook-Pro T04 % ./q3
Final value of sum = 48418
ctank@Colins-MacBook-Pro T04 % ./q3
Final value of sum = 97265
ctank@Colins-MacBook-Pro T04 % ./q3
Final value of sum = 75757
ctank@Colins-MacBook-Pro T04 % ./q3
Final value of sum = 82678
ctank@Colins-MacBook-Pro T04 % ./q3
Final value of sum = 63670
ctank@Colins-MacBook-Pro T04 %
```

This is caused by race-conditions on the global variable sum. If you recall, almost all architectures are load-store architectures. In such architectures operations can only be performed on registers, and any variable must be loaded to a register first.

Thus sum++ is implemented in assembly using three instructions:

```
LD $r0, sum          # Load sum from memory into register $r0
ADD $r0, r0, 1        # Increment $r0
ST $r0, sum           # Write result back to sum
```

Suppose sum is currently 0. With two threads updating, there are several possible “interleavings” of these three instructions:

First Interleaving - Correct

Thread 1	Thread 2	Description
LD \$r0, sum		Load sum to \$r0, \$r0 = 0
ADD \$r0, \$r0, 1		\$r0 = 1
ST \$r0, sum		Write \$r0 to sum. sum=1
	LD \$r0, sum	Load sum to \$r0. \$r0=1
	ADD \$r0, \$r0, 1	\$r0 = 2
	ST \$r0, sum	Write \$r0 to sum. sum = 2

Second interleaving – Wrong

Thread 1	Thread 2	Description
LD \$r0, sum		Load sum to \$r0, \$r0 = 0
	LD \$r0, sum	Load sum to \$r0, \$r0 = 0
ADD \$r0, \$r0, 1		\$r0 = 1
	ADD \$r0, \$r0, 1	\$r0 = 1 (See note)
ST \$r0, sum		Write \$r0 to sum, sum=1
	ST \$r0, sum	Write \$r0 to sum, sum=1

Note: Although thread 1 has updated \$r0 to 1, \$r0 remains 0 when Thread 2 runs because of context switching. Hence add \$r0, \$r0, 1 results in a 1.

We now see how to fix this.

Question 4. Fixing Race Conditions

We will now use “mutual exclusions” or mutexes to fix race conditions. A mutex is a lock that can be held by at most one thread using “pthread_mutex_lock”. If the mutex is locked by one thread, any other thread calling pthread_mutex_lock will block until the holding thread calls “pthread_mutex_unlock”.

Create a file called q4.c, and key in the following code:

We begin first by including our usual header files, and declare a mutex lock of type pthread_mutex_t. We also have our shared variable sum:

```
#include <stdio.h>
#include <pthread.h>

pthread_mutex_t mutex;

int sum = 0;
```

We now update our addone thread function by locking the mutex, updating sum, then unlocking the mutex:

```
void *addone(void *p) {
    for(int i=0; i<10000; i++){
        pthread_mutex_lock(&mutex);
        sum++;
        pthread_mutex_unlock(&mutex);
    }

    pthread_exit(NULL);
}
```

Our main is almost the same, except that we initialize the mutex by calling “pthread_mutex_init”. The second parameter lets us configure the mutex. We will use the defaults and hence pass in NULL.

We also destroy the mutex once we are done:

```
int main() {  
  
    int i;  
  
    pthread_t tid[10];  
  
    pthread_mutex_init(&mutex, NULL);  
    for(i=0; i<10; i++)  
        pthread_create(&tid[i], NULL, addone, NULL);  
  
    for(i=0; i<10; i++)  
        pthread_join(tid[i], NULL);  
  
    printf("Final value of sum = %d\n", sum);  
  
    pthread_mutex_destroy(&mutex);  
  
    return 0;  
}
```

Compile our program using:

```
gcc -pthread q4.c -o q4
```

We run using:

```
./q4
```

We now see that no matter how many times we run this program, it will always give the correct answer 100,000:

```
ctank@Colins-MacBook-Pro T04 % ./q4  
Final value of sum = 100000  
ctank@Colins-MacBook-Pro T04 % ./q4  
Final value of sum = 100000  
ctank@Colins-MacBook-Pro T04 % ./q4  
Final value of sum = 100000  
ctank@Colins-MacBook-Pro T04 %
```

Why does this work? Let's examine what happens in this interleaving:

Thread 1	Thread 2	Description
pthread_mutex_lock		Thread 1 locks the mutex
	pthread_mutex_lock	Thread 2 blocks because thread 1 is holding the mutex
LD \$r0, sum		Load sum to \$r0, \$r0=0
ADD \$r0, \$r0, 1		\$r0=1
ST \$r0, sum		Write \$r0 to sum, sum = 1
pthread_mutex_unlock		Unlock the mutex, allowing Thread 2 to run.
	LD \$r0, sum	Load sum to \$r0, \$r0=1
	ADD \$r0, \$r0, 1	\$r0=2
	ST \$r0, sum	Store \$r0 to sum, sum=2

Notice how Thread 2 blocks when it tries to acquire the mutex, because Thread 1 has already acquired it. Thread 1 can then update sum fully, and when it unlocks the thread, Thread 2 can now read the correct value of 1 (instead of the stale value of 0), and update it correctly to 2.

Summary

This tutorial shows you quickly how to write pthread programs, and how to solve race conditions using mutual exclusions, or mutexes.